# 第 22 章：工具集成与 API 设计

**版本**: v2.0（润色版）
**作者**: 内容撰写专家（上下游篇）
**状态**: 润色完成
**最后更新**: 2026-03-23

---

## 【本章导读】

**学习目标**： - 掌握 RESTful API 设计原则 - 理解 Function Calling 的实现机制 - 能够设计 Webhook 事件驱动和 OAuth 认证流程

**核心内容**： 工具集成是 Agent 系统的「手脚」，决定了 Agent 能做什么。本章从 RESTful API 设计、Function Calling、Webhook 事件驱动、OAuth 认证四个维度，系统讲解如何设计和集成工具。

---

## 22.1 RESTful API 设计

RESTful（Representational State Transfer，表述性状态转移，一种 API 设计风格）API 是工具集成的基础，良好的 API 设计能显著提升系统的可维护性和可扩展性。

### 一、REST 原则深度解析

**无状态（Stateless）**：

每个请求必须包含所有必要信息，服务器不保存会话状态。

```
❌ 有状态设计： 请求 1: POST /login {username, password} → 服务器保存 session 请求 2: GET /comics/1 → 服务器从 session 读取用户信息

✅ 无状态设计： 请求 1: POST /login {username, password} → 返回 token 请求 2: GET /comics/1 (Header: Authorization: Bearer ) → 从 token 解析用户信息
```

**为什么需要无状态**： - **可扩展**：任何服务器实例都能处理任何请求，支持水平扩展 - **可靠性**：单点故障不影响整体，请求可路由到健康实例 - **简化设计**：服务器不需要管理会话状态，降低复杂度

**统一接口（Uniform Interface）**：

资源用 URL 标识，操作用 HTTP 方法表示。

```` ❌非 RESTful 设计： GET /getComic?id=1 POST /createComic POST /updateComic?id=1 POST /deleteComic?id=1

✅RESTful 设计： GET /comics/1 # 获取 POST /comics # 创建 PUT /comics/1 # 更新 DELETE /comics/1 # 删除 ```

**可缓存（Cacheable）**：

响应可标注是否可缓存，减少重复请求。

```http

# 可缓存响应

Cache-Control: public, max-age=3600 ETag: "abc123"

# 不可缓存响应

Cache-Control: no-store ```

**漫剧案例应用**： 漫剧发布 API 遵循 REST 原则： - 无状态：每次请求携带 API Key（API 密钥，用于身份认证）或 JWT（JSON Web Token，轻量级认证令牌） - 统一接口： `GET /comics/{id}`、`POST /comics`、`PUT /comics/{id}` - 可缓存：漫剧详情响应缓存 1 小时（`max-age=3600`），列表不缓存

**常见误区**： ❌在 URL 中包含动作（如 `/getComic`）——实际应该用 `/comics/{id}` + GET 方法。 ❌用 POST 做所有操作——实际应该根据语义选择正确的 HTTP 方法。

---

## 二、资源设计规范

**资源命名**： - **用名词复数**： `/comics` 而非 `/comic` - **小写字母**： `/user-profiles` 而非 `/UserProfiles` - **连字符分隔**： `/user-profiles` 而非 `/user_profiles` 或 `/userProfiles` - **避免动词**：资源是名词，动作用 HTTP 方法表示

**嵌套资源**： `GET /comics/{id}/chapters # 获取某漫剧的所有章节 GET /comics/{id}/chapters/{num} # 获取某漫剧的第 N 章 POST /comics/{id}/chapters # 为某漫剧创建新章节`

**嵌套深度限制**： 建议不超过 3 层，过深说明资源设计可能有问题。 ✅ 合理：`/comics/{id}/chapters/{num}/sections` ❌ 过深：`/users/{id}/projects/{pid}/tasks/{tid}/comments/{cid}/replies`

**过滤与排序**：```

# 过滤

GET /comics?genre=fantasy&status=published

# 排序

GET /comics?sort=created_at&order=desc

# 组合

GET /comics?genre=fantasy&sort=created_at&order=desc&page=1&limit=20 ```

**分页设计**：```json // 请求 GET /comics?page=1&limit=20

// 响应 { "data": [...], "pagination": { "total": 150, "page": 1, "limit": 20, "total_pages": 8, "has_next": true, "has_prev": false } } ```

**分页策略对比**：

| 策略 | 优点 | 缺点 | 适用场景 | |------|------|------|---------| | **页码分页**（page/limit） | 简单直观，可跳页 | 深度分页性能差 | 通用场景 | | **游标分页**（cursor/limit） | 性能好，数据一致 | 不能跳页 | 大数据量、实时数据 | | **时间戳分页**（since/limit） | 简单，适合增量 | 只能顺序访问 | 日志、消息流 |

**漫剧案例**：漫剧系统资源设计：`/comics # 漫剧列表 /comics/{id} # 漫剧详情 /comics/{id}/chapters # 章节列表 /comics/{id}/chapters/{num} # 章节详情 /characters # 角色列表 /characters/{id} # 角色详情 /settings # 设定列表 /settings/{type}/{id} # 特定类型设定`

**常见误区**：❌资源名用动词——实际应该用名词，动作用 HTTP 方法表示。❌嵌套过深——实际超过 3 层应该考虑扁平化设计。

---

## 三、状态码规范

**2xx 成功**：- **200 OK**：请求成功，返回数据（GET/PUT） - **201 Created**：资源创建成功（POST），Location Header 指向新资源 - **204 No Content**：请求成功，无返回内容（DELETE）

**4xx 客户端错误**： - **400 Bad Request**：请求格式错误、参数验证失败 - **401 Unauthorized**：未认证或认证失败 - **403 Forbidden**：已认证但无权限 - **404 Not Found**：资源不存在 - **409 Conflict**：资源冲突（如重复创建） - **422 Unprocessable Entity**：语义错误（参数合法但业务不允许） - **429 Too Many Requests**：限流

**5xx 服务器错误**： - **500 Internal Server Error**：服务器内部错误 - **502 Bad Gateway**：上游服务错误 - **503 Service Unavailable**：服务暂时不可用（维护/过载） - **504 Gateway Timeout**：上游服务超时

**错误响应格式**： `json { "error": { "code": "VALIDATION_ERROR", "message": "参数验证失败", "details": [ { "field": "title", "message": "标题不能为空" }, { "field": "genre", "message": "类型必须是 fantasy/urban/romance 之一" } ], "request_id": "req_abc123" } }`

**漫剧案例**： ```

# 创建漫剧成功

POST /comics → 201 Created Location: /comics/123

# 漫剧不存在

GET /comics/999 → 404 Not Found

# 未授权

DELETE /comics/123 (无 Token) → 401 Unauthorized

# 无权限

DELETE /comics/123 (非作者) → 403 Forbidden

# 参数错误

POST /comics {title: ""} → 400 Bad Request

# 限流

POST /comics (频繁请求) → 429 Too Many Requests ```

**常见误区**： ❌所有错误都返回 200 + 错误信息——实际应该用正确状态码，便于客户端处理。 ❌滥用 500——实际 500 是「未知错误」，已知错误应该用具体状态码。

---

## 22.2 Function Calling

Function Calling 让 LLM 能够可靠地调用工具，是 Agent 系统的核心机制。

### 一、LLM 原生工具调用机制

**定义与原理**：

Function Calling 不是真正的函数调用，而是 LLM 输出结构化的调用意图，由框架解析后执行。

```传统方式： 用户：「查一下北京天气」 → LLM 理解意图 → 自由文本输出「北京天气晴朗...」 → 难以解析，可靠性低

Function Calling： 用户：「查一下北京天气」 → LLM 输出：{"name": "get_weather", "arguments": {"city": "北京"}} → 框架解析 → 执行 get_weather("北京") → 返回结果 → LLM 整合结果 → 输出「北京天气晴朗...」 ```

**为什么需要 Function Calling**： - **可靠性高**：结构化输出，易于解析和验证 - **可组合**：多个工具可串联调用 - **可追溯**：记录每次调用的输入输出，便于调试 - **安全**：LLM 只能调用预定义的工具，不能执行任意代码

**支持模型**： - **OpenAI**：GPT-3.5/4（Function Calling 原生支持） - **Anthropic**：Claude 3（Tool Use） - **Google**：Gemini（Function Calling） - **开源模型**：需要微调或 Prompt 工程模拟

**工作流程**： `1. 定义函数（名称、描述、参数） ↓ 2. 将函数定义发送给 LLM（作为 System Prompt 一部分） ↓ 3. LLM 决定是否需要调用函数 ↓ ├─ 不需要 → 直接输出回复 └─ 需要 → 输出函数调用意图（JSON 格式） ↓ 4. 框架解析 JSON，执行对应函数 ↓ 5. 将函数返回结果发送给 LLM ↓ 6. LLM 整合结果，输出最终回复`

**漫剧案例应用**： 漫剧设定检索定义为函数： `json { "name": "search_setting", "description": "检索漫剧设定，支持角色、世界观、剧情类型", "parameters": { "type": "object", "properties": { "query": { "type": "string", "description": "搜索关键词，如角色名、设定术语" },`
```

`"type": { "type": "string", "enum": ["character", "world", "plot"], "description": "设定类型" } }, "required": ["query"] } }`

当用户问「主角轩辕墨的能力是什么」时： 1. LLM 输出： `{"name": "search_setting", "arguments": {"query": "轩辕墨", "type": "character"}}` 2. 框架执行检索，返回角色设定 3. LLM 整合设定，输出回复

**常见误区**： ❌ 认为「Function Calling 是真正的函数调用」——实际是 LLM 输出结构化意图，框架执行。 ❌ 函数定义过于简略——实际 LLM 靠描述决定是否调用，需要详细准确。

---

## 二、函数定义与描述

**函数命名**： - **简洁有意义**： `search_setting` 而非 `func1` 或 `doSomething` - **动词 + 名词**： `get_weather` 、 `create_comic` 、 `update_chapter` - **一致性**：相似功能用相似命名（ `get_xxx` 、 `list_xxx` 、 `create_xxx` ）

**函数描述**： 描述质量直接影响 LLM 选择准确性。

```` ❌ 简略描述： "搜索设定"

✅ 详细描述： "检索漫剧设定数据库，支持按角色名、世界观术语、剧情关键词搜索。 返回匹配的设定详情，包括名称、描述、属性等。 适用于回答用户关于角色能力、世界规则、剧情背景的问题。" ```

**参数定义**：
`json { "name": "generate_chapter", "description": "生成漫剧章节正文", "parameters": { "type": "object", "properties": { "comic_id": { "type": "string", "description": "漫剧 ID" }, "chapter_number": { "type": "integer", "description": "章节编号，从 1 开始", "minimum": 1 }, "plot_points": { "type": "array", "items": {"type": "string"}, "description": "本章需要覆盖的剧情点列表" }, "style": { "type": "string", "enum": ["action", "dialogue", "narrative"], "description": "文风偏好", "default": "narrative" } }, "required": ["comic_id", "chapter_number", "plot_points"] } }`

**参数描述技巧**： - **说明用途**：不只是「是什么」，还要说明「用来做什么」 - **提供示例**： `"description": "城市名，如'北京'、'上海'"` - **标注约束**： `"minimum": 1` 、 `"maxLength": 100` - **默认值**：可选参数设置 `"default": "value"`

**多函数场景**： 当有多个函数时，LLM 需要选择调用哪个。
````

`json [ { "name": "search_setting", "description": "检索已有设定（只读）" }, { "name": "create_setting", "description": "创建新设定（需要用户确认）" }, { "name": "update_setting", "description": "修改已有设定（需要用户确认）" } ]`

**漫剧案例**：漫剧系统函数库（部分）：`search_setting # 检索设定 create_setting # 创建设定 generate_outline # 生成大纲 generate_chapter # 生成章节 check_consistency # 检查设定一致性 publish_comic # 发布漫剧`

**常见误区**：❌ 函数描述太简略——实际 LLM 靠描述决定是否调用，需要详细。❌ 参数描述不清晰——实际 LLM 可能提取错误参数值。

---

## 三、参数提取与验证

**参数提取**：

LLM 从对话中提取参数值。

`用户：「帮我生成第 3 章，要包含主角觉醒和反派登场」 → LLM 提取： { "chapter_number": 3, "plot_points": ["主角觉醒", "反派登场"] }`

**缺失参数处理**：

当必要参数缺失时，追问用户。

`用户：「帮我生成一章」 → LLM 发现缺少 chapter_number 和 plot_points → 回复：「请问要生成第几章？本章需要包含哪些剧情点？」`

**实现方式**：```python def handle_function_call(function_name, arguments): # 检查必要参数 required_params = get_required_params(function_name) missing = [p for p in required_params if p not in arguments]

```
if missing:
    # 追问用户
    return ask_user_for_missing_params(missing)

# 参数完整，执行函数
return execute_function(function_name, arguments)
```

**格式验证**：

检查参数类型、范围、格式。

```python def validate_params(function_name, arguments): errors = []

# 类型检查
if not isinstance(arguments.get("chapter_number"), int):
    errors.append("chapter_number 必须是整数")

# 范围检查
if arguments.get("chapter_number", 0) < 1:
    errors.append("chapter_number 必须 >= 1")

# 枚举检查
if arguments.get("style") not in ["action", "dialogue", "narrative"]:
    errors.append("style 必须是 action/dialogue/narrative 之一")

return errors

```

**错误反馈**:

验证失败时告知用户具体错误。

```❌ 模糊错误: 「参数错误，请重新输入」

✅ 具体错误: 「章节编号必须是正整数，您输入的是'第一页'。请问是第 1 章吗？」 ```

**参数转换**:

用户输入可能需要转换。

`用户：「第一页」→ 转换为 1 用户：「下周五」→ 转换为日期 2026-03-27 用户：「全部」→ 转换为 ["action", "dialogue", "narrative"]`

**漫剧案例**: 漫剧检索参数验证: ```python def validate_search_params(query, type=None): if not query or len(query.strip()) == 0: return "搜索关键词不能为空"

```
if type and type not in ["character", "world", "plot"]:
    return "类型必须是 character/world/plot 之一"

if len(query) > 100:
    return "搜索关键词不能超过 100 字"

return None  # 验证通过
```

**常见误区**: ❌ 不验证直接用——实际 LLM 可能提取错误，导致后续失败。 ❌ 错误信息不具体——实际用户不知道如何修正。

---
```

# 22.3 Webhook 与事件驱动

Webhook 实现事件驱动的通知机制，是异步任务处理的关键组件。

## 一、Webhook 原理深度解析

**定义**：

Webhook 是事件发生时主动推送通知到指定 URL 的机制（回调）。

```
轮询模式：
客户端：「有更新吗？」→ 服务端：「没有」
客户端：「有更新吗？」→ 服务端：「没有」
客户端：「有更新吗？」→ 服务端：「有」
（大部分请求是无效的）

Webhook 模式：
服务端：[事件发生] → 主动推送通知 → 客户端
（只在有事件时通信）
```

**与轮询对比**：

| 维度 | 轮询 | Webhook |
|---|---|---|
| **实时性** | 取决于轮询间隔（通常 3-5 秒） | 实时（事件发生立即通知） |
| **服务器压力** | 高（大部分请求无更新） | 低（只在有事件时请求） |
| **客户端复杂度** | 简单（定时请求） | 中等（需要接收端点） |
| **网络要求** | 客户端能访问服务端 | 服务端能访问客户端（公网 URL） |
| **适用场景** | 低频事件、内网环境 | 高频事件、实时性要求高 |

**优势**：
- **实时性好**：事件发生立即通知，无需等待
- **减少无效请求**：只在有事件时通信
- **解耦**：事件生产者和消费者解耦

**劣势**：
- **需要公网 URL**：客户端必须有服务端可访问的端点
- **需要处理重试**：网络波动可能导致通知失败
- **需要签名验证**：防止伪造请求

**漫剧案例应用**：漫剧章节生成完成时，Webhook 通知前端页面刷新，用户无需手动刷新或轮询。

`用户提交生成请求 → 后台处理 → 生成完成 → Webhook 通知前端 → 前端刷新显示结果`

**常见误区**：❌ 认为「Webhook 一定比轮询好」——实际低频事件用轮询更简单。❌ 不处理通知失败——实际需要重试机制确保送达。

---

## 二、事件订阅与通知

**事件类型定义**：

`json { "events": [ { "name": "comic.created", "description": "漫剧创建成功" }, { "name": "chapter.completed", "description": "章节生成完成" }, { "name": "setting.updated", "description": "设定修改" }, { "name": "publish.succeeded", "description": "发布成功" }, { "name": "publish.failed", "description": "发布失败" } ] }`

**订阅管理**:

用户选择订阅哪些事件、通知 URL。

```json // 订阅请求 POST /webhooks/subscriptions { "url": "https://example.com/webhook", "events": ["chapter.completed", "publish.succeeded"], "secret": "whsec_abc123" }

// 订阅响应 { "id": "sub_xyz789", "url": "https://example.com/webhook", "events": ["chapter.completed", "publish.succeeded"], "status": "active", "created_at": "2026-03-22T10:00:00Z" } ```

**通知格式**:

统一的事件格式，便于处理。

`json { "id": "evt_abc123", "type": "chapter.completed", "created_at": "2026-03-22T10:30:00Z", "data": { "comic_id": "comic_123", "chapter_number": 3, "status": "completed", "word_count": 3500 }, "request_id": "req_xyz789" }`

**重试机制**:

通知失败时重试。

```python def send_webhook(url, event, max_retries=5): for attempt in range(max_retries): try: response = requests.post( url, json=event, headers={"X-Webhook-Signature": sign(event)}, timeout=10 ) if response.status_code == 200: return True

```
        # 4xx 错误不重试（客户端错误）
        if 400 <= response.status_code < 500:
            log error(f"Webhook 客户端错误：{response.status_code}")
            return False

    except Exception as e:
        log_error(f"Webhook 发送失败：{e}")

    # 指数退避
    wait time = (2 ** attempt) + random.uniform(0, 1)
    time.sleep(wait_time)
```

`return False`

```

**重试策略**： - **最大重试次数**：3-5 次 - **退避策略**：指数退避（1s, 2s, 4s, 8s, 16s） - **放弃后处理**：记录失败事件，支持手动重发

**漫剧案例**： 漫剧系统支持订阅 `chapter.completed` 事件： `1. 用户在平台订阅 Webhook，URL: https://myapp.com/webhook 2. 章节生成完成，系统发送通知 3. 用户服务器接收通知，更新数据库，推送前端 4. 返回 200 确认接收 5. 如失败，系统按指数退避重试，最多 5 次`

**常见误区**： ❌ 不处理通知失败——实际需要重试机制确保送达。 ❌ 不验证签名——实际可能被伪造请求攻击。

---

## 三、异步任务处理

**问题**：

耗时任务（如生成长文本）不能阻塞请求。

```
❌ 同步处理： POST /comics/1/chapters/generate → 等待 60 秒生成完成 → 返回结果 （HTTP 请求超时，用户体验差）

✅ 异步处理： POST /comics/1/chapters/generate → 立即返回任务 ID → 后台处理 → 用户轮询状态或等待 Webhook 通知
```

**方案：任务队列 + 后台 Worker**：

`客户端 ──→ API 服务 ──→ 任务队列 ──→ Worker ──→ 执行任务 │ │ ↓ ↓ 返回任务 ID 完成时 Webhook 通知`

**任务状态**： `json { "task_id": "task_abc123", "status": "pending", "progress": 0, "result": null, "error": null, "created_at": "2026-03-22T10:00:00Z", "updated_at": "2026-03-22T10:00:00Z" }`

**状态查询**： `GET /tasks/task_abc123 → { "task_id": "task_abc123", "status": "running", "progress": 50, "result": null, "error": null }`

**结果通知**： - **Webhook**：任务完成时主动通知 - **WebSocket**：实时推送进度 - **轮询**：客户端定期查询状态
```

**漫剧案例**：漫剧章节生成异步处理：`1. 用户 POST /comics/1/chapters/generate 2. API 返回 {task_id: "task_123", status: "pending"} 3. 任务进入队列，Worker 拾取执行 4. 用户可 GET /tasks/task_123 查询进度 5. 生成完成，Webhook 通知用户 6. 用户 GET /comics/1/chapters/3 查看结果`

**常见误区**：❌ 长任务同步处理——实际会导致请求超时。❌ 不提供进度查询——实际用户需要知道任务进展。

---

## 22.4 OAuth 与认证

认证和授权是保护 API 安全的关键机制。

### 一、OAuth 2.0 流程深度解析

**OAuth（Open Authorization，开放授权，行业标准授权协议）** 是一种授权协议，允许第三方应用有限访问用户资源，而无需获取用户密码。

**角色定义**：- **资源所有者（Resource Owner）**：用户 - **客户端（Client）**：第三方应用 - **授权服务器（Authorization Server）**：颁发令牌 - **资源服务器（Resource Server）**：提供 API

**授权码模式（Authorization Code Flow）**：

`1. 用户点击「用 Google 登录」 ↓ 2. 浏览器跳转到 Google 授权页面 ↓ 3. 用户同意授权 ↓ 4. Google 重定向回客户端，带授权码 ↓ 5. 客户端用授权码换取访问令牌 ↓ 6. 客户端用访问令牌访问 Google API`

**详细流程**：

```
Step 1: 客户端引导用户到授权服务器
GET https://accounts.google.com/o/oauth2/auth?
client_id=YOUR_CLIENT_ID&
redirect_uri=YOUR_REDIRECT_URI&
response_type=code&
scope=email profile&
state=random_state_string

Step 2: 用户同意授权

Step 3: 授权服务器重定向回客户端
GET YOUR_REDIRECT_URI?
code=AUTHORIZATION_CODE&
state=random_state_string

Step 4: 客户端用授权码换取令牌
POST https://oauth2.googleapis.com/token
grant_type=authorization_code&
code=AUTHORIZATION_CODE&
redirect_uri=YOUR_REDIRECT_URI&
client_id=YOUR_CLIENT_ID&
client_secret=YOUR_CLIENT_SECRET
```

Step 5: 返回访问令牌 { "access_token": "ya29.abc123...", "token_type": "Bearer", "expires_in": 3600, "refresh_token": "1//abc123..." }

Step 6: 用访问令牌访问资源 GET https://www.googleapis.com/oauth2/v2/userinfo Authorization: Bearer ya29.abc123... ```

**令牌类型**： - **访问令牌（Access Token）**：短期（通常 1 小时），用于访问资源 - **刷新令牌（Refresh Token）**：长期（通常数月），用于刷新访问令牌

**刷新令牌**： ```POST https://oauth2.googleapis.com/token grant_type=refresh_token& refresh_token=REFRESH_TOKEN& client_id=YOUR_CLIENT_ID& client_secret=YOUR_CLIENT_SECRET

→ 返回新的 access_token ```

**为什么需要 OAuth**： - **用户不直接给应用密码**：而是授权有限访问权限 - **可撤销**：用户可随时撤销授权 - **细粒度**：可授权部分权限（如只读邮箱，不能修改）

**漫剧案例应用**： 漫剧平台允许用户用 Google 账号登录： 1. 用户点击「用 Google 登录」 2. 跳转 Google 授权 3. 授权后获取用户邮箱和头像 4. 用邮箱创建或关联漫剧平台账号

**常见误区**： ❌ 认为「OAuth 就是单点登录」——实际 OAuth 是授权协议，不是认证协议（OIDC 才是认证）。 ❌ 不验证 state 参数——实际可能导致 CSRF 攻击。

---

## 二、API Key 管理

**生成**：

用加密随机数生成。

```python import secrets

# 生成 32 字节（64 字符）的 API Key

api_key = secrets.token_hex(32) ```

**存储**：

服务端哈希存储，不存明文。

```python import hashlib

# 存储时哈希

stored_hash = hashlib.sha256(api_key.encode()).hexdigest()

# 验证时

def verify_api_key(provided_key, stored_hash): provided_hash = hashlib.sha256(provided_key.encode()).hexdigest() return secrets.compare_digest(provided_hash, stored_hash) ```

**传递**:

HTTP Header 传递。

```http Authorization: Bearer

# 或

X-API-Key: ```

**轮换**:

定期更换 Key，旧 Key 设置宽限期。

`1. 生成新 API Key 2. 同时允许旧 Key 和新 Key 访问（宽限期 7 天） 3. 通知用户更新 4. 宽限期后禁用旧 Key`

**漫剧案例**： 漫剧 API 用 API Key 认证开发者访问： - Key 存储在环境变量，不写入代码 - 每次请求验证 Key 有效性 - 支持 Key 轮换，旧 Key 宽限期 7 天

**常见误区**： ❌ API Key 写死在代码中——实际应该用环境变量或密钥管理服务。 ❌ 不哈希存储——实际泄露后无法挽回。

---

## 三、用户授权

**权限粒度**:

`读权限（read）：查看资源 写权限（write）：修改资源 删除权限（delete）：删除资源 管理权限（admin）：管理用户和权限`

**授权范围（Scope）**：
```

`scope: comic:read comic:write chapter:read → 可读写漫剧，只读章节`

**授权撤销**：

用户可随时撤销授权，令牌立即失效。

`DELETE /oauth/tokens/{token_id} → 令牌立即失效，后续请求返回 401`

**审计日志**：

记录谁在什么时候访问了什么资源。

`json { "timestamp": "2026-03-22T10:30:00Z", "user_id": "user_123", "action": "chapter.read", "resource": "comics/1/chapters/3", "ip": "192.168.1.100", "user_agent": "Mozilla/5.0..." }`

**漫剧案例**：漫剧协作平台权限设计：`作者：comic:read, comic:write, chapter:read, chapter:write, delete 编辑：comic:read, chapter:read, chapter:write 读者：comic:read, chapter:read`

作者可授权编辑修改章节，但不能删除漫剧。

**常见误区**：❌ 授权后不记录日志——实际无法追溯谁做了什么操作。❌ 权限过于粗放——实际应该细粒度控制，最小权限原则。

---

# 22.5 简单举例

## 案例设计

- **案例名称**：漫剧发布的 API 集成
- **涉及知识点**：RESTful API 设计、Function Calling、Webhook 与事件驱动、OAuth 与认证
- **案例目标**：帮助理解如何设计一套 API 集成方案实现漫剧的自动化发布和状态同步
- **案例内容要点**：
- 场景描述：漫剧完成后需要发布到多个平台（网站、APP、第三方平台），需要设计 API 集成方案实现自动化发布和状态同步
- 技术应用：设计 RESTful 发布接口异步处理，用 Function Calling 让 Agent 决定何时发布，Webhook 通知发布结果，OAuth 认证第三方平台访问
- 效果说明：发布流程自动化用户只需确认一次，Agent 完成所有平台的发布和状态同步，发布结果实时通知，第三方平台访问安全可控
- **注意事项**：不展开 OAuth 的具体实现细节（见 22.4 节）

---

## 22.6 最佳实践与陷阱

**最佳实践**： 1. **API 设计先行**：先设计 API 接口，再实现功能 2. **文档即代码**：API 文档与代码同步更新（如 OpenAPI Spec） 3. **版本管理**：API 版本化（`/v1/comics`），支持平滑升级 4. **限流从第一天开始**：即使初期流量小，也要实现限流 5. **日志完整记录**：记录每次 API 调用的请求、响应、耗时

**常见陷阱**：

1. **陷阱 1：API 设计随意变更**
2. 问题：API 接口频繁变更，客户端适配困难
3. 避免：版本化管理，旧版本至少支持 6 个月
4. **陷阱 2：Function Calling 不验证参数**
5. 问题：LLM 提取的参数可能错误，直接使用导致失败
6. 避免：严格验证参数，缺失时追问用户
7. **陷阱 3：Webhook 不重试**
8. 问题：网络波动导致通知丢失
9. 避免：实现重试机制，最多 5 次，指数退避
10. **陷阱 4：OAuth 令牌泄露**
11. 问题：令牌写入日志或客户端代码
12. 避免：令牌只存服务端，日志脱敏
13. **陷阱 5：忽略幂等性**
14. 问题：重复请求导致重复操作（如重复扣费）
15. 避免：写操作支持幂等（用唯一 ID 去重）

---

**知识来源**: - REST API 设计最佳实践 - https://restfulapi.net/ - OpenAI Function Calling 官方文档 - https://platform.openai.com/docs/guides/function-calling - OAuth 2.0 RFC 6749 - https://datatracker.ietf.org/doc/html/rfc6749

---

**修改记录**: - v2.0 (2026-03-23): 润色版 — 句子简化、删除重复、优化结构、统一语气 - v1.1 (2026-03-22): 根据编辑统筹意见修改 — 规范知识来源格式 - v1.0 (2026-03-22): 初稿完成